

Automatización inteligente del flujo de pedidos para un mercado de frutas y verduras

Esp. Ing. Joaquín Manuel Beitia

Maestría en Computación de Borde

Director: Matías Martini (Xepelin)

Jurados:

Jurado 1 (pertenencia)

Jurado 2 (pertenencia)

Jurado 3 (pertenencia)

Ciudad de Rosario, julio de 2026

Resumen

El presente trabajo consiste en el diseño e implementación de un sistema que automatiza la recepción, interpretación y el procesamiento de los pedidos de un mercado de frutas y verduras recibidos a través de mensajería instantánea. El sistema captura pedidos en texto, voz o imagen, e identifica productos, cantidades y unidades mediante modelos de inteligencia artificial. Una vez estructurado el pedido, se imprime en el comercio mediante una impresora térmica conectada a un servidor local. El desarrollo fue realizado para la Tienda de frutas Luján, con el fin de reducir los tiempos operativos, disminuir los errores humanos y mejorar la trazabilidad de la gestión comercial.

La solución se estructura sobre una arquitectura de cómputo distribuida entre el comercio y la nube, en la que las tareas críticas y de baja latencia se ejecutan localmente, mientras que las de mayor demanda de procesamiento se delegan a servicios remotos. El sistema se encuentra operativo en el comercio, donde procesa los pedidos en tiempo real y deja cada pedido listo para su preparación en menos de un minuto desde su recepción. Para su desarrollo se aplicaron conocimientos propios de la computación de borde, tales como la distribución del procesamiento entre el borde y la nube, la comunicación segura entre nodos, la actuación local sobre hardware y el diseño de soluciones de bajo costo.

Agradecimientos

Quiero expresar mi agradecimiento a la Tienda de frutas Luján, y en particular a Marcos y Cristian Cestari, por la confianza depositada en este trabajo y por abrir las puertas de su comercio para el desarrollo y la validación del sistema. Su predisposición y su conocimiento del negocio fueron fundamentales para orientar el trabajo hacia una solución realmente útil.

Agradezco también a mi director, Matías Martini, cuya orientación y experiencia en desarrollo de software e inteligencia artificial aplicada guiaron las decisiones técnicas a lo largo del desarrollo.

Mi reconocimiento se extiende a mis compañeros y colegas, cuya colaboración fue esencial para superar los desafíos surgidos durante la implementación. Por último, agradezco a mis amigos y a mi familia, cuya paciencia y aliento incondicional fueron mi sostén durante este proceso, permitiéndome perseguir mis proyectos y crecer profesionalmente.

Índice general

Resumen	I
1. Introducción general	1
1.1. Mensajería instantánea y gestión de pedidos en comercios minoristas	1
1.2. Contexto y motivación	2
1.3. Estado del arte	3
1.3.1. Soluciones basadas en la nube	3
1.3.2. Plataformas de automatización de mensajería	3
1.3.3. Enfoques de borde e híbridos	4
1.3.4. Comparación y posicionamiento del trabajo	4
1.4. Objetivos del trabajo	4
2. Introducción específica	7
2.1. Dispositivos de borde	7
2.1.1. Servidor local	7
2.1.2. Impresora térmica	8
2.2. Protocolos de comunicación	8
2.2.1. Canal de mensajería	9
2.2.2. Comunicación segura con la nube	9
2.3. Plataformas cloud y servicios	10
2.3.1. Orquestación de flujos de trabajo	10
2.3.2. Infraestructura de despliegue	10
2.3.3. Servicios de inteligencia artificial	10
2.4. Herramientas de almacenamiento y visualización	12
2.4.1. Bases de datos	12
2.4.2. Servidor local y panel de visualización	12
3. Diseño e implementación	13
3.1. Arquitectura del sistema	13
3.1.1. Criterio de distribución del cómputo	14
3.1.2. Organización del código	14
3.1.3. Entorno de desarrollo	14
3.1.4. Flujo de datos extremo a extremo	15
3.2. Capa de captura y comunicación	15
3.2.1. Recepción de los mensajes	15
3.2.2. Clasificación y procesamiento por tipo de contenido	15
3.2.3. Agrupación de mensajes fragmentados	16
3.3. Capa de red y comunicaciones	17
3.3.1. Tramos de comunicación	17
3.3.2. Vinculación con el nodo de borde	18
3.3.3. Formato de los mensajes intercambiados	18
3.4. Capa de aplicación y procesamiento	19

3.4.1.	Interpretación mediante agentes de inteligencia artificial . . .	19
3.4.2.	Normalización semántica de productos	20
3.4.3.	Recursos de la interfaz de programación	21
3.4.4.	Modelo de datos	21
3.4.5.	Resolución de precios	22
3.4.6.	Generación del ticket	23
3.5.	Interfaz de usuario y visualización	23
3.5.1.	El ticket impreso	23
3.5.2.	El panel web de control	23
	Tablero principal	25
	Gestión de pedidos	25
	Administración del catálogo y de los precios	26
	Métricas de operación	26
3.6.	Seguridad del sistema	29
3.6.1.	Autenticación y control de acceso	29
3.6.2.	Protección de las comunicaciones	30
3.6.3.	Gestión de credenciales e integridad de los datos	30
4.	Ensayos y resultados	31
4.1.	Pruebas funcionales del hardware	31
5.	Conclusiones	33
5.1.	Conclusiones generales	33
5.2.	Próximos pasos	33
	Bibliografía	35

Índice de figuras

1.1. Esquema conceptual del sistema: capas, componentes y flujo de datos.	2
3.1. Arquitectura del sistema en tres capas.	13
3.2. Procesamiento de la capa de captura, desde el mensaje entrante hasta el texto agrupado.	16
3.3. Topología de despliegue del sistema.	18
3.4. Diagrama entidad-relación de la base de datos.	22
3.5. Ticket generado por el sistema e impreso en el comercio.	24
3.6. Tablero principal del panel de control.	25
3.7. Listado de pedidos con sus filtros de búsqueda.	26
3.8. Detalle de un pedido, con edición de ítems y acceso al mensaje original.	27
3.9. Administración del catálogo de productos.	27
3.10. Administración de las listas de precios de un cliente.	28
3.11. Vista de métricas de operación del comercio.	28
3.12. Mecanismos de autenticación para personas y para servicios internos.	29

Índice de tablas

1.1. Comparación de paradigmas para la automatización de pedidos en el comercio minorista.	4
2.1. Modelos de inteligencia artificial empleados en cada etapa de la interpretación.	11
3.1. Grupos de recursos de la interfaz de programación.	21

A mi familia.

Capítulo 1

Introducción general

En el presente capítulo se introduce el área en la que se enmarca el trabajo, se describe el contexto y la motivación que dieron origen al desarrollo, se revisan las soluciones existentes para problemas similares y se enuncian los objetivos perseguidos. El problema abordado es la gestión manual de los pedidos que los comercios reciben por canales de mensajería, y su resolución resulta relevante por el volumen de operaciones que hoy se procesan de esa forma.

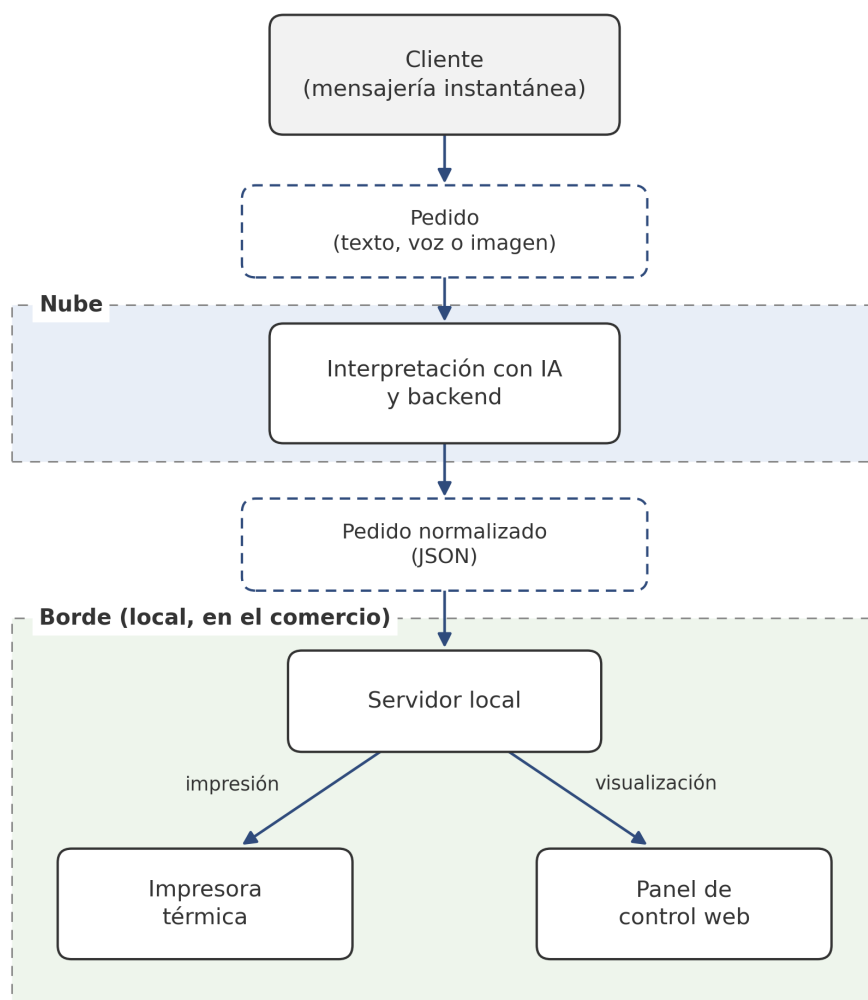
1.1. Mensajería instantánea y gestión de pedidos en comercios minoristas

La creciente adopción de canales de mensajería instantánea como medio de contacto comercial modificó la forma en que los pequeños y medianos comercios reciben los pedidos de sus clientes. En el rubro de los mercados de frutas y verduras, gran parte de las ventas se gestiona a través de aplicaciones de mensajería, donde los pedidos llegan en formatos heterogéneos: texto libre, mensajes de voz, fotografías de listas manuscritas o capturas de pantalla. La interpretación de estos mensajes y su conversión en un pedido preparable constituyen, hoy, una tarea manual realizada por el personal del comercio.

El trabajo se sitúa en la intersección entre la automatización de procesos comerciales y la computación de borde (*edge computing*), entendida esta última como el paradigma en el que parte del procesamiento y la toma de decisiones se ejecutan cerca del lugar donde se generan y se consumen los datos, en lugar de delegarse íntegramente a la nube. En un comercio minorista, el “borde” es el propio local: allí se recibe el pedido, allí debe imprimirse el ticket para su armado y allí ocurre la operación que no puede detenerse ante una caída de conectividad. Este enfoque contrasta con las arquitecturas puramente basadas en la nube, en las que todo el procesamiento se centraliza en servidores remotos.

El sistema desarrollado combina componentes de inteligencia artificial (IA) para la interpretación de los mensajes con una infraestructura de Internet de las cosas (IoT, del inglés *Internet of Things*) para la actuación física en el local, articulados mediante una arquitectura híbrida que distribuye el cómputo entre un servidor local en el comercio y servicios alojados en la nube. El foco de esta memoria está puesto en los aspectos propios de la computación de borde: la distribución del procesamiento entre el borde y la nube, la comunicación segura entre ambos, la actuación local sobre hardware y las restricciones de costo, latencia y disponibilidad características de estos entornos. Los fundamentos de aprendizaje de máquina y aprendizaje profundo se asumen conocidos y no se desarrollan en detalle.

En la figura 1.1 se presenta un esquema conceptual de alto nivel del sistema, en el que el pedido ingresa por el canal de mensajería, se interpreta en la nube y deriva en la impresión local del ticket y en la visualización de la operación. La arquitectura detallada del sistema se describe en el capítulo 3.



Trazo continuo: componentes del sistema.

Trazo punteado: datos que circulan entre los componentes.

FIGURA 1.1. Esquema conceptual del sistema: capas, componentes y flujo de datos.

1.2. Contexto y motivación

El trabajo se desarrolló para la Tienda de frutas Luján, un comercio minorista de frutas y verduras que recibe sus pedidos principalmente a través de mensajería instantánea. En la operación actual, un integrante del negocio lee cada mensaje entrante, interpreta su contenido (que puede llegar como texto, audio o imagen), normaliza los nombres y las cantidades de los productos y transcribe el pedido a un formato que permita su preparación y despacho.

Este proceso manual presenta varias limitaciones. En primer lugar, consume tiempo del personal que podría destinarse a la atención y a la preparación de los pedidos, y su capacidad se satura en los horarios de mayor demanda. En segundo lugar, la transcripción manual es propensa a errores de interpretación, especialmente ante mensajes de voz extensos, listas manuscritas poco legibles o denominaciones ambiguas de un mismo producto. En tercer lugar, al no existir un registro estructurado de los pedidos, se pierde trazabilidad: resulta difícil reconstruir el historial de un cliente, auditar un pedido puntual o extraer información agregada para la gestión comercial.

La motivación del desarrollo es, entonces, doble. Por un lado, se busca reducir los tiempos operativos y los errores humanos mediante la automatización de la recepción, interpretación e impresión de los pedidos. Por el otro, se procura hacerlo con una solución accesible y de bajo costo, adecuada a la realidad de un comercio que no dispone de infraestructura tecnológica avanzada ni de personal técnico especializado. Esta última condición es la que justifica el enfoque de computación de borde: procesar y actuar localmente permite que la operación crítica (la impresión del ticket para armar el pedido) funcione con baja latencia y sin depender por completo de la disponibilidad de servicios remotos.

1.3. Estado del arte

La automatización de la recepción de pedidos en el comercio minorista puede abordarse desde distintos paradigmas, que se diferencian principalmente por dónde se ejecuta el procesamiento y por el grado de adaptación al canal de mensajería informal. A continuación, se revisan las alternativas existentes y se las contrasta con el enfoque adoptado en este trabajo.

1.3.1. Soluciones basadas en la nube

Una parte importante de las soluciones comerciales de gestión de pedidos y punto de venta se ofrece bajo el modelo de software como servicio, con todo el procesamiento centralizado en la nube. Entre las plataformas más conocidas se encuentran Shopify [1] y, en el mercado local, Tiendanube [2], que ofrecen soluciones de punto de venta y gestión de pedidos. Estas plataformas ofrecen paneles de administración, reportes y, en algunos casos, integración con canales de mensajería. Su principal ventaja es que no requieren infraestructura local más allá de un dispositivo con conexión a Internet. Sus limitaciones, en el contexto de este trabajo, son la dependencia total de la conectividad para operar, los costos recurrentes de suscripción y la escasa capacidad de personalización frente a un flujo de pedidos informal y no estructurado como el que se recibe por mensajería.

1.3.2. Plataformas de automatización de mensajería

Existen plataformas y herramientas de automatización de flujos de trabajo que permiten conectar canales de mensajería con otros servicios mediante reglas o disparadores, como n8n [3] o Zapier [4]. Estas soluciones habilitan respuestas automáticas y la orquestación de tareas, pero por sí solas no resuelven la interpretación semántica de un pedido expresado en lenguaje natural, en audio o en una

imagen. Para ello deben combinarse con modelos de procesamiento de lenguaje, reconocimiento de voz y reconocimiento óptico de caracteres (OCR, del inglés *Optical Character Recognition*).

1.3.3. Enfoques de borde e híbridos

Frente a los esquemas puramente centralizados, la computación de borde propone ejecutar parte del procesamiento cerca del origen de los datos [5, 6]. En el comercio minorista, este enfoque resulta atractivo cuando existe una actuación física local, como la impresión de un ticket, que debe ocurrir con baja latencia y tolerancia a fallos de conectividad. Las arquitecturas híbridas, que combinan un nodo local con servicios en la nube, permiten equilibrar la capacidad de cómputo de la nube con la inmediatez y la resiliencia del procesamiento local [7].

1.3.4. Comparación y posicionamiento del trabajo

En la tabla 1.1 se comparan los tres paradigmas descriptos según los criterios más relevantes para el caso de uso abordado. El trabajo desarrollado se posiciona en el enfoque híbrido. En la nube se ejecutan las tareas de mayor demanda de cómputo, como la interpretación de los mensajes mediante modelos de IA. En el borde se ejecuta la actuación crítica y de baja latencia, es decir, la impresión local del pedido. De este modo, se busca un compromiso entre costo, personalización y disponibilidad, adecuado a un comercio de baja complejidad tecnológica.

TABLA 1.1. Comparación de paradigmas para la automatización de pedidos en el comercio minorista.

Criterio	Nube pura	Automatización de mensajería	Enfoque híbrido (este trabajo)
Procesamiento local	No	Parcial	Sí (actuación local)
Dependencia de conectividad	Alta	Alta	Media
Personalización del flujo	Baja	Media	Alta
Costo de infraestructura	Bajo	Bajo	Bajo/Medio
Trazabilidad estructurada	Variable	Baja	Alta

1.4. Objetivos del trabajo

El objetivo general del trabajo fue diseñar e implementar un sistema de automatización de pedidos. El sistema abarca la recepción, la interpretación y el procesamiento de los pedidos de un mercado de frutas y verduras recibidos a través de mensajería. De este modo se buscó reducir los tiempos operativos, minimizar los errores humanos y mejorar la trazabilidad de la operación comercial.

Para alcanzarlo, se definieron los siguientes objetivos específicos, formulados de manera concreta y verificable:

- Capturar de forma automática los pedidos entrantes por el canal de mensajería, en sus distintos formatos: texto, voz e imagen.
- Interpretar el contenido de cada pedido mediante modelos de inteligencia artificial, para identificar productos, cantidades y unidades, y normalizar denominaciones equivalentes de un mismo producto.
- Generar, a partir de esa interpretación, un pedido en formato estructurado y persistirlo de manera que quede registrada la trazabilidad completa, desde la recepción hasta la impresión.
- Imprimir automáticamente el pedido en una impresora térmica conectada a un servidor local, para garantizar la actuación en el borde con baja latencia.
- Establecer una comunicación segura entre el servidor local y los servicios en la nube.
- Proveer un panel web que permita visualizar los pedidos en tiempo real, el historial y métricas básicas de operación.
- Validar el sistema en condiciones reales de operación en el comercio.

El alcance del trabajo se limita al flujo de recepción, interpretación, impresión y visualización de pedidos. Las tecnologías y componentes de terceros empleados en la construcción del sistema se describen en el capítulo [2](#), mientras que el diseño y la implementación de la arquitectura se detallan en el capítulo [3](#).

Capítulo 2

Introducción específica

En el presente capítulo se describen las tecnologías, plataformas, servicios y componentes de terceros utilizados como base para el desarrollo del sistema. Se presentan los dispositivos de borde, los protocolos de comunicación, las plataformas en la nube, los servicios de inteligencia artificial, las herramientas de almacenamiento y los recursos de visualización empleados en la solución.

2.1. Dispositivos de borde

El nodo de borde del sistema está constituido por un servidor local instalado en el comercio y por una impresora térmica conectada a él. A diferencia de un sistema de Internet de las Cosas tradicional, no se emplean microcontroladores ni sensores. El dispositivo de borde es un equipo de cómputo de propósito general que ejecuta los servicios locales y comanda la impresión de los pedidos.

2.1.1. Servidor local

Como servidor local se utiliza una computadora con sistema operativo Windows Server 2019 [8], propiedad del cliente y preexistente en el local. Sobre este equipo se ejecutan el servidor de impresión y el cliente del túnel seguro hacia la nube.

Se evaluaron dos alternativas para el nodo de borde. La primera consistió en incorporar una computadora de placa única (SBC, del inglés *Single Board Computer*) dedicada, como una Raspberry Pi [9]. La segunda consistió en reutilizar el equipo ya disponible en el comercio. Se optó por la segunda opción a partir de tres criterios.

El primer criterio fue el costo. El trabajo se planteó como una solución económica para un comercio pequeño. La reutilización del hardware disponible elimina la inversión marginal en equipamiento. El segundo criterio fue la operación. El equipo se encuentra en el local, permanece encendido durante el horario comercial y resulta conocido por el personal. Esto simplifica la instalación y el mantenimiento posterior. El tercer criterio fue la compatibilidad. La impresora térmica seleccionada provee controladores para Windows, de modo que la integración no requirió desarrollos adicionales de bajo nivel.

Como contrapartida, un equipo de propósito general consume más energía que una SBC dedicada y puede verse condicionado por otros usos dentro del comercio. Esta limitación se consideró aceptable frente a la simplicidad operativa obtenida.

También se descartó el uso de un microcontrolador. Un dispositivo de este tipo puede ejecutar funciones simples, como un servidor web básico o la comunicación con periféricos. Sin embargo, el conjunto de responsabilidades del nodo de borde (un servidor web local, el cliente del túnel seguro y la administración de la cola de impresión) se resuelve de manera más conveniente sobre un sistema operativo completo.

2.1.2. Impresora térmica

Para la impresión de los tickets se utiliza una impresora térmica Hasar P-HAS-181 [10], un modelo de tipo comandera del fabricante argentino Grupo Hasar, orientado al punto de venta. La impresora imprime por cabezal térmico directo sobre rollos de papel de 80 mm de ancho, con corte automático de papel y resolución de 203 dpi (del inglés, *dots per inch*, puntos por pulgada). Admite la impresión de códigos de barras y códigos QR (del inglés, *Quick Response*). Dispone de tres interfaces de conexión (puerto serie, USB y Ethernet) y de controladores para Windows. Sus comandos de impresión resultan compatibles con ESC/POS [11], el conjunto de instrucciones definido por Epson que constituye el estándar de facto del sector.

En la instalación actual, la impresora se conecta al servidor local por USB. La disponibilidad de una interfaz Ethernet fue, sin embargo, un criterio de selección. El equipo puede requerir un cambio de ubicación dentro del comercio, por ejemplo hacia el sector de armado de pedidos. La conexión por red permitiría ese traslado sin dependencia de la proximidad física con el servidor.

La elección de este modelo se fundamentó en cuatro factores. Primero, la impresión térmica es el estándar del comercio minorista por su velocidad y su bajo costo operativo, dado que no utiliza tinta ni cartuchos. Segundo, la compatibilidad con ESC/POS permite comandar el equipo desde bibliotecas de software ampliamente disponibles, sin dependencia de un protocolo propietario. Tercero, la disponibilidad de controladores para Windows garantiza la integración directa con el servidor local. Cuarto, se trata de un fabricante nacional con presencia establecida en el mercado argentino, lo que asegura la provisión local de insumos, repuestos y servicio técnico. Este último criterio resulta relevante para un comercio sin personal técnico propio.

2.2. Protocolos de comunicación

La comunicación del sistema no se apoya en los protocolos habituales de las redes de sensores, como MQTT o LoRaWAN, sino en protocolos y servicios de la web. El intercambio de mensajes entre los distintos componentes se realiza mediante HTTP (del inglés, *Hypertext Transfer Protocol*), un protocolo de nivel de aplicación basado en peticiones y respuestas, definido en el estándar RFC 9110 [12]. Las interfaces expuestas siguen el estilo arquitectónico REST (del inglés, *Representational State Transfer*), formulado por Fielding [13], que organiza la comunicación alrededor de recursos identificados por direcciones y operaciones uniformes sobre ellos.

Esta elección responde a que todos los servicios involucrados exponen sus funcionalidades a través de interfaces de programación de aplicaciones (API, del inglés

Application Programming Interface) basadas en HTTP. Para los eventos asincrónicos, como la llegada de un mensaje, se emplea el mecanismo de notificaciones automáticas (*webhooks*), incorporado como elemento de primer nivel en la versión 3.1 de la especificación OpenAPI [14]. Este mecanismo invierte el sentido habitual de la comunicación: el servicio que origina el evento emite una petición hacia una dirección configurada por el sistema receptor, sin que este último deba consultar de forma periódica por novedades.

2.2.1. Canal de mensajería

La vinculación con WhatsApp se realiza mediante Evolution API [15], una pasarela de mensajería de código abierto que expone las funciones de la plataforma a través de una interfaz REST y de notificaciones automáticas. Evolution API admite dos modos de conexión. El primero se basa en el protocolo de WhatsApp Web, mediante la biblioteca Baileys, y no requiere verificación comercial ante Meta. El segundo se basa en la API oficial WhatsApp Business Cloud [16]. En este trabajo se utiliza el primer modo. Ante cada mensaje entrante, de texto, voz o imagen, la pasarela emite una notificación con el contenido o con las referencias para descargarlo.

La elección de Evolution API para la etapa inicial del proyecto se fundamentó en tres razones: su costo nulo, la rapidez de puesta en marcha (no requiere el proceso de verificación comercial ni la aprobación de plantillas de la API oficial) y su despliegue autoalojado, coherente con la infraestructura contenerizada del sistema. Su principal limitación reside en que el modo basado en WhatsApp Web constituye un mecanismo no oficial, con posibles restricciones de estabilidad y de cumplimiento frente a la alternativa provista por Meta. Por este motivo, el sistema se diseñó de manera compatible con la API de WhatsApp Business y se encuentra prevista la migración a la vía oficial. Esta compatibilidad se estableció como requerimiento de interoperabilidad desde la planificación del proyecto, de modo que el cambio no implica modificaciones estructurales en los flujos de procesamiento.

2.2.2. Comunicación segura con la nube

Para vincular los servicios en la nube con el servidor local sin apertura de puertos en la red del comercio, se emplea un túnel seguro provisto por Cloudflare [17]. Un cliente ligero, ejecutado en el servidor local, establece una conexión cifrada saliente hacia la infraestructura del proveedor. A través de esa conexión, los servicios remotos alcanzan al nodo de borde.

Las alternativas consideradas presentaban desventajas claras para este escenario. La exposición directa del servidor mediante la apertura de puertos en el enrutador del local requiere una dirección IP pública o un servicio de DNS dinámico (del inglés, *Domain Name System*), exige configurar el equipamiento de red del comercio y deja un servicio a la escucha de conexiones entrantes desde Internet, lo que amplía la superficie de ataque. Una red privada virtual (VPN, del inglés *Virtual Private Network*) entre la nube y el local resuelve la seguridad, pero agrega administración de claves y configuración en ambos extremos. El túnel de Cloudflare, en cambio, no requiere configuración alguna del enrutador, ya que la conexión

es saliente. Además, cifra el tráfico de extremo a extremo y mantiene la red interna del negocio sin exposición entrante. Todo ello con una operación mínima, condición determinante en un local sin personal técnico.

2.3. Plataformas cloud y servicios

2.3.1. Orquestación de flujos de trabajo

La orquestación de las tareas de recepción, interpretación y estructuración de los pedidos se realiza con n8n [3], una plataforma de automatización de flujos de trabajo de código abierto. En n8n, la lógica se construye mediante la conexión de nodos configurables (disparadores, transformaciones e integraciones con servicios externos) en flujos visuales. Este enfoque permite integrar servicios heterogéneos sin desarrollar desde cero la lógica de integración. La plataforma incorpora además agentes de inteligencia artificial (*AI Agents*), que encadenan modelos de lenguaje con herramientas y fuentes de datos. Sobre esa capacidad se apoya la interpretación de los pedidos.

Frente a alternativas comerciales de automatización en la nube, como Zapier [4], n8n presentó dos ventajas decisivas para este trabajo. La primera es la posibilidad de autoalojamiento en la propia infraestructura, sin costo por tarea ejecutada, un factor crítico dado el volumen diario de mensajes. La segunda es que los datos de los clientes permanecen dentro del entorno controlado por el sistema, sin atravesar una plataforma de terceros adicional.

2.3.2. Infraestructura de despliegue

Los servicios en la nube se alojan en un servidor privado virtual (VPS, del inglés *Virtual Private Server*) del proveedor Hostinger [18], en su plan KVM 4. Este plan provee 4 núcleos virtuales de procesamiento, 16 GB de memoria RAM, 200 GB de almacenamiento NVMe (del inglés, *Non-Volatile Memory Express*) y 16 TB de transferencia mensual. El dimensionamiento respondió a la carga concurrente del sistema. Sobre el mismo servidor conviven la pasarela de mensajería, el motor de flujos, las bases de datos y el panel web. El almacenamiento NVMe favorece la baja latencia de las operaciones sobre la base de datos.

Todos los servicios se despliegan de forma contenerizada mediante Docker [19]. La contenerización empaqueta cada servicio junto con sus dependencias en unidades aisladas y reproducibles, lo que aporta tres beneficios concretos. El despliegue completo del sistema se describe en archivos versionables. El entorno puede reproducirse en otro servidor, o en un equipo local de pruebas, sin diferencias de configuración. Cada componente puede actualizarse o reiniciarse de manera independiente, sin afectar al resto.

2.3.3. Servicios de inteligencia artificial

La interpretación de los pedidos requiere resolver tres tareas diferenciadas: convertir los mensajes de voz en texto, leer las listas manuscritas enviadas como fotografía y extraer de un texto informal los productos, las cantidades y las unidades solicitadas. Cada tarea se resuelve con un modelo de inteligencia artificial consumido como servicio en la nube, provisto por OpenAI [20] o por Anthropic [21].

La transcripción de audio se realiza con el servicio de transcripción de OpenAI, basado en el modelo Whisper [22], un sistema de reconocimiento automático del habla entrenado sobre un corpus extenso y multilingüe. Su característica más relevante para este trabajo es la robustez frente a condiciones adversas de grabación: ruido de fondo, variabilidad de acentos y calidad desigual del micrófono. Los mensajes de voz de los clientes se graban dentro del comercio o en la vía pública, sin control sobre el entorno sonoro, por lo que esa robustez resultó determinante.

La lectura de las imágenes se resuelve con modelos multimodales de visión. Se evaluó como alternativa un motor de reconocimiento óptico de caracteres (OCR, del inglés *Optical Character Recognition*) clásico, que transcribe texto a partir de la forma de los caracteres. Las imágenes recibidas presentan una variedad amplia: listas manuscritas fotografiadas con el teléfono, con caligrafías dispares e iluminación irregular, y también capturas de planillas de cálculo enviadas como imagen. En ese escenario, el OCR tradicional pierde precisión, porque opera sobre cada carácter de manera aislada y sin considerar la disposición del contenido. Los modelos multimodales, en cambio, interpretan la imagen en su contexto, reconocen la estructura tabular y aprovechan el conocimiento del dominio para resolver ambigüedades.

Sobre cada imagen se ejecutan dos modelos de proveedores distintos: GPT-4o de OpenAI y Claude Sonnet de Anthropic. La decisión respondió a un problema detectado durante el desarrollo: ante una misma imagen, ambos modelos producen en ocasiones lecturas discrepantes, en particular con caligrafías difíciles o con planillas de estructura compleja. Las dos lecturas se reconcilian luego en un tercer paso, donde un modelo de lenguaje las compara, corrige los errores detectados y devuelve una transcripción única. Esta redundancia agrega costo y latencia, pero reduce el riesgo de un error silencioso en la etapa más frágil de la interpretación.

La estructuración final se resuelve con un modelo extenso de lenguaje (LLM, del inglés *Large Language Model*) de OpenAI, integrado como agente dentro del flujo. El modelo recibe el texto del pedido, ya sea el original o el obtenido de las etapas anteriores, junto con el catálogo de productos del comercio, y produce una representación estructurada con los productos, las cantidades y las unidades. Esta capacidad de interpretar lenguaje informal, con abreviaturas, errores de tipeo y denominaciones no normalizadas, constituye el aporte central de estos modelos a la solución.

En la tabla 2.1 se resumen los modelos empleados en cada etapa.

TABLA 2.1. Modelos de inteligencia artificial empleados en cada etapa de la interpretación.

Etapa	Modelo	Proveedor
Transcripción de audio	Whisper	OpenAI
Lectura de imagen (primera)	GPT-4o	OpenAI
Lectura de imagen (segunda)	Claude Sonnet	Anthropic
Reconciliación de lecturas	GPT-5.5	OpenAI
Estructuración del pedido	GPT-5.5	OpenAI

La decisión de consumir estos modelos como servicio, en lugar de ejecutarlos sobre la propia infraestructura, define la distribución del cómputo entre el borde y la nube. Los tres modelos exigen recursos de procesamiento muy superiores a

los del nodo de borde, por lo que su ejecución local no resultaba viable con el equipamiento del comercio. Su consumo como servicio mantiene la inversión inicial y la complejidad operativa en el mínimo. El costo es doble: un componente variable por uso y la dependencia de la conectividad para la etapa de interpretación. La actuación crítica del sistema permanece en el borde, de manera que esa dependencia no compromete la operación local de impresión.

2.4. Herramientas de almacenamiento y visualización

2.4.1. Bases de datos

La información de pedidos, clientes y productos se almacena en PostgreSQL [23], una base de datos relacional de código abierto, madura y ampliamente adoptada. Su modelo relacional con integridad referencial resulta adecuado para el dominio del problema, donde los pedidos se vinculan con clientes y con artículos del catálogo. La trazabilidad exigida al sistema, es decir la reconstrucción del recorrido de cada pedido desde el mensaje original hasta la impresión, requiere garantías de consistencia transaccional. Las consultas estructuradas sobre el historial alimentan además las métricas del panel de control.

De manera complementaria se emplea Redis [24], una base de datos en memoria de tipo clave-valor, para la gestión de las sesiones y del estado de los clientes activos durante la recepción de un pedido. Redis ofrece tiempos de acceso muy bajos y expiración automática de claves, características apropiadas para datos de vida corta con alta frecuencia de lectura y escritura. La combinación responde a un criterio de diseño explícito: los datos persistentes y auditables residen solo en la base relacional, mientras que Redis se reserva para información temporal. De este modo, una eventual falla de la capa en memoria no compromete los registros del negocio.

2.4.2. Servidor local y panel de visualización

El servidor de impresión que se ejecuta en el nodo de borde se implementó sobre Flask [25], un entorno de desarrollo web minimalista escrito en el lenguaje Python. Flask permite exponer servicios HTTP con una huella de recursos reducida y sin la complejidad de un servidor de aplicaciones completo. Resulta adecuado para un servicio local de propósito específico, que recibe el pedido estructurado y comanda la impresora sobre el equipo Windows del comercio.

La visualización de los pedidos, del historial y de las métricas de operación se ofrece a través de un panel web alojado en el VPS, accesible desde cualquier navegador moderno. Este enfoque evita la instalación de aplicaciones adicionales en los dispositivos del comercio. El panel se desarrolló con Angular [26], un entorno de desarrollo de aplicaciones web mantenido por Google, junto con la biblioteca de componentes Angular Material. La representación de las métricas se resuelve con Chart.js [27]. La elección de un entorno estructurado, con tipado estático y organización por componentes, respondió a la necesidad de sostener el crecimiento del panel a medida que se incorporan nuevas vistas y funciones de administración.

Capítulo 3

Diseño e implementación

En el presente capítulo se describen el diseño y la implementación del sistema. Se presenta la arquitectura general y la distribución del cómputo entre el comercio y la nube. Luego se detalla cada capa: la captura de los pedidos, la conectividad entre los componentes, el procesamiento y la persistencia, la interfaz de usuario y las medidas de seguridad. En cada sección se exponen los problemas encontrados, los criterios adoptados y la justificación de las decisiones tomadas.

3.1. Arquitectura del sistema

El sistema se organizó en tres capas funcionales. La capa de captura y comunicación recibe los mensajes del canal de mensajería y los prepara para su interpretación. La capa de procesamiento inteligente interpreta el contenido, lo estructura contra el catálogo del comercio y lo persiste. La capa de interacción local e impresión ejecuta la actuación física sobre la impresora del negocio. En la figura 3.1 se presentan las tres capas, sus componentes y el flujo de datos extremo a extremo.

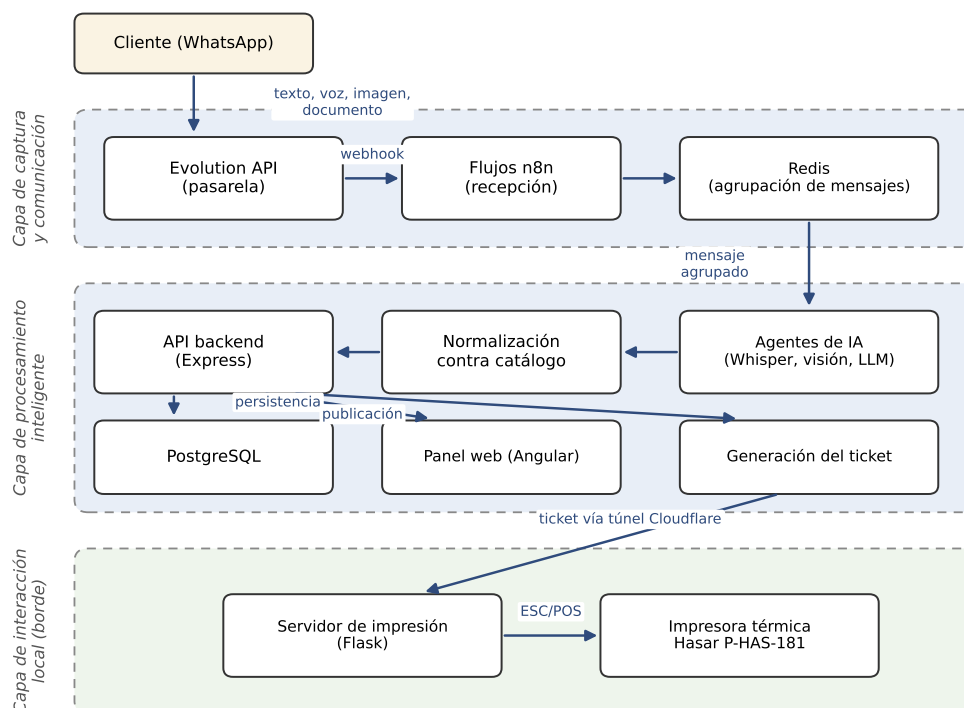


FIGURA 3.1. Arquitectura del sistema en tres capas.

3.1.1. Criterio de distribución del cómputo

El criterio rector del diseño fue asignar cada tarea al lugar de ejecución que mejor equilibra latencia, disponibilidad y costo. Las tareas de mayor demanda de procesamiento son la transcripción de voz, la lectura de imágenes y la estructuración mediante modelos de lenguaje. Todas ellas se delegaron a servicios en la nube, donde la capacidad de cómputo resulta elástica y los modelos se consumen bajo demanda. Ejecutarlas sobre el equipo del comercio habría exigido una inversión en hardware incompatible con las restricciones económicas del proyecto.

La actuación crítica del negocio es la impresión del ticket que habilita el armado del pedido. Esa tarea se resolvió en el borde, sobre el servidor local del comercio. La decisión responde a tres motivos. El primero es la latencia: la orden de impresión recorre un único salto desde el servidor local hasta la impresora. El segundo es la independencia del canal: la impresora nunca se expone a Internet, sino que queda detrás de un servicio local que actúa como intermediario. El tercero es la continuidad operativa: el personal puede reimprimir un pedido ya recibido desde el panel sin que intervengan los modelos de inteligencia artificial.

3.1.2. Organización del código

El código fuente se organizó en cuatro módulos independientes, que se corresponden con los componentes de la arquitectura:

- `api/`: la interfaz de programación de aplicaciones (API) y la lógica de pedidos, implementadas con Node.js, Express y TypeScript sobre el mapeador objeto-relacional (ORM, del inglés *Object-Relational Mapping*) Prisma.
- `web/panel/`: el panel web de control, desarrollado con Angular.
- `n8n/`: los flujos de automatización que orquestan la recepción y la interpretación de los pedidos.
- `print_server/`: el servidor de impresión que se ejecuta en el nodo de borde, implementado con Flask.

Esta separación permite desplegar, actualizar y probar cada módulo por separado. La API y el panel se comunican solo a través de recursos HTTP, sin dependencias de código compartido, de modo que un cambio en la interfaz no obliga a modificar el servidor.

3.1.3. Entorno de desarrollo

El desarrollo planteó una dificultad práctica: la impresora se encuentra en el comercio y permanece en uso durante el horario comercial, por lo que no resultaba viable ensayar contra el equipo real cada modificación del flujo.

La solución adoptada fue una versión simulada del servidor de impresión. Ese componente expone el mismo recurso y acepta el mismo formato de petición que el servidor real, pero en lugar de enviar el ticket a la impresora lo registra en la consola. El archivo de composición de contenedores del entorno de desarrollo levanta la API, el panel y esa versión simulada, y dirige las peticiones de impresión hacia ella de forma automática.

El resultado fue un entorno de desarrollo completo, reproducible en cualquier equipo y sin dependencia del hardware del comercio. La sustitución se resuelve

por configuración, sin cambios en el código de la API, lo que garantiza que el componente ejercitado durante las pruebas sea el mismo que opera en producción.

3.1.4. Flujo de datos extremo a extremo

El recorrido de un pedido comprende las siguientes etapas. El cliente envía uno o varios mensajes por WhatsApp. La pasarela Evolution API los entrega al flujo de n8n mediante una notificación automática. El flujo clasifica cada mensaje según su tipo y aplica el procesamiento correspondiente hasta obtener texto plano. Ese texto se acumula en una lista temporal en Redis, asociada al teléfono del cliente, y se agrupa con los mensajes contiguos del mismo remitente. El conjunto agrupado se entrega al agente de inteligencia artificial junto con el catálogo vigente de productos y unidades. El agente devuelve el pedido estructurado. La API lo valida, resuelve los precios aplicables, lo persiste y solicita la impresión. El servidor local recibe el ticket a través del túnel seguro y lo envía a la impresora. En paralelo, el pedido queda disponible en el panel web para su seguimiento.

3.2. Capa de captura y comunicación

Esta capa recibe los mensajes de los clientes y los convierte en un texto único, listo para su interpretación. Constituye la entrada del sistema y concentra buena parte de su complejidad, porque debe absorber la heterogeneidad del canal.

3.2.1. Recepción de los mensajes

La recepción se resolvió mediante el mecanismo de notificaciones automáticas de la pasarela Evolution API. Ante cada mensaje entrante, la pasarela emite una petición HTTP hacia el punto de entrada del flujo de n8n. El primer nodo del flujo normaliza la estructura del mensaje recibido y extrae los datos relevantes: el teléfono del remitente, su nombre de perfil, el identificador del mensaje, el tipo de contenido y la marca temporal.

El teléfono cumple una función central en el diseño, ya que opera como identificador natural del cliente a lo largo de todo el flujo. Se utiliza como clave de agrupación de mensajes, como clave de sesión del agente y como criterio de búsqueda del cliente en la base de datos, donde el campo correspondiente tiene restricción de unicidad. La elección resultó natural para el dominio: el número de teléfono es estable, lo provee el propio canal sin intervención del cliente y no requiere ningún registro previo.

El nombre de perfil de WhatsApp, en cambio, resultó insuficiente para identificar al cliente en el negocio. Muchas personas configuran un apodo o dejan el campo vacío, y el comercio conoce a sus clientes por su nombre real o por su razón social. Por ese motivo, el flujo complementa la identificación con una consulta a la agenda de contactos asociada al comercio, y la base de datos conserva un campo propio con el nombre que el personal administra desde el panel.

3.2.2. Clasificación y procesamiento por tipo de contenido

Un mismo pedido puede llegar como texto libre, como uno o varios mensajes de voz, como una fotografía de una lista manuscrita, como una captura de una

planilla o como un documento adjunto. El flujo resuelve esa diversidad con un nodo de derivación que evalúa el tipo de contenido y envía el mensaje a la rama correspondiente. En la figura 3.2 se muestra el procesamiento completo de esta capa.

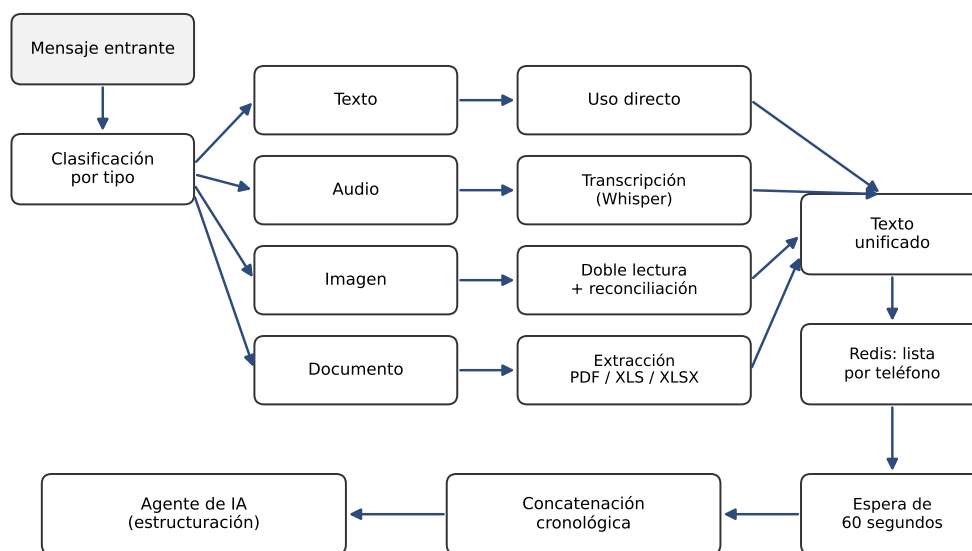


FIGURA 3.2. Procesamiento de la capa de captura, desde el mensaje entrante hasta el texto agrupado.

Los mensajes de texto se emplean de forma directa. Los mensajes de voz y las imágenes requieren un paso previo de descarga: la pasarela entrega el contenido multimedia codificado en base64, que luego se convierte en un archivo binario antes de enviarlo al modelo correspondiente. Los mensajes de voz se transcriben con el servicio de transcripción de OpenAI. Los documentos adjuntos se derivan a una rama específica, que reconoce el formato y extrae su contenido mediante los nodos de extracción de PDF y de planillas de cálculo.

El tratamiento de las imágenes constituye el caso más elaborado. La imagen se envía en paralelo a dos modelos de proveedores distintos, GPT-4o de OpenAI y Claude Sonnet de Anthropic. Ambas lecturas se combinan y se entregan a un tercer modelo, que las compara, corrige las diferencias y devuelve una transcripción única.

La instrucción entregada a los modelos de visión no se limita a transcribir el texto visible. Incorpora reglas del dominio que surgieron de la operación real. La más relevante indica conservar la estructura de las tablas y descartar las filas cuya cantidad pedida sea cero. Esa regla resolvió un caso concreto: algunos clientes mayoristas envían una planilla con el catálogo completo del comercio y completan solo unas pocas filas. Sin esa instrucción, el modelo transcribía el listado íntegro y el agente generaba un pedido con decenas de productos no solicitados.

3.2.3. Agrupación de mensajes fragmentados

El problema más relevante de esta capa no es la interpretación de un mensaje, sino la determinación de dónde termina un pedido. En el uso real, un cliente rara

vez envía su pedido en un único mensaje. Lo habitual es una secuencia de mensajes cortos, con mezcla de texto y audio, enviados en el transcurso de un minuto. Si el sistema procesara cada mensaje por separado, generaría varios pedidos incompletos en lugar de uno correcto.

La solución adoptada consiste en un mecanismo de agrupación por ventana temporal, implementado sobre Redis. Cada texto obtenido se agrega al final de una lista cuya clave es el teléfono del cliente. A continuación, el flujo espera un tiempo determinado. Vencido ese plazo, recupera la lista completa, descarta las entradas inválidas, ordena los elementos por su marca temporal y los concatena en un texto único. La lista se elimina luego para dejar la sesión limpia. Los mensajes que llegan durante la espera se acumulan en la misma lista y quedan incluidos en la agrupación, sin disparar un procesamiento adicional.

La elección de Redis para este mecanismo responde a la naturaleza de los datos involucrados. Se trata de información de vida corta, con alta frecuencia de escritura y lectura, cuya pérdida ante una falla no compromete la operación del negocio. La base relacional se reservó para los datos que sí deben persistir y auditarse.

La ventana se fijó en sesenta segundos. El valor no se eligió a partir del comportamiento de los clientes, sino del tiempo de procesamiento del propio sistema: el intervalo entre el ingreso de un mensaje y la finalización de su procesamiento por los agentes resultó cercano al minuto. Una ventana menor habría cerrado la agrupación antes de que los mensajes previos terminaran de procesarse, con el riesgo de fragmentar el pedido. El valor adoptado constituye entonces una cuota conservadora, alineada con la latencia observada. La reducción de ese tiempo de procesamiento permitiría acortar la ventana y, con ella, el tiempo total de respuesta del sistema. Esa optimización se plantea como línea de trabajo futuro en el capítulo 5.

3.3. Capa de red y comunicaciones

En la figura 3.3 se presenta la topología de despliegue del sistema, con los servicios contenerizados del servidor virtual y los componentes instalados en el comercio.

3.3.1. Tramos de comunicación

La conectividad del sistema comprende tres tramos con características distintas.

El primero es el tramo de entrada. La pasarela Evolution API, alojada en el servidor virtual, mantiene la vinculación con la red de WhatsApp y notifica cada mensaje al flujo de n8n mediante peticiones HTTPS, la variante cifrada de HTTP.

El segundo es el tramo interno de la nube. Los servicios contenerizados se comunican entre sí dentro del mismo servidor: el flujo de n8n consulta el catálogo a la API, la API accede a PostgreSQL, y el panel web consume los mismos recursos de la API que utiliza el flujo. Junto a estos servicios opera Chatwoot, una plataforma de gestión de conversaciones que centraliza la bandeja de entrada de WhatsApp. Su integración permite que el personal acceda desde el panel al hilo original de la conversación que dio origen a un pedido, lo que resulta útil ante una interpretación dudosa.

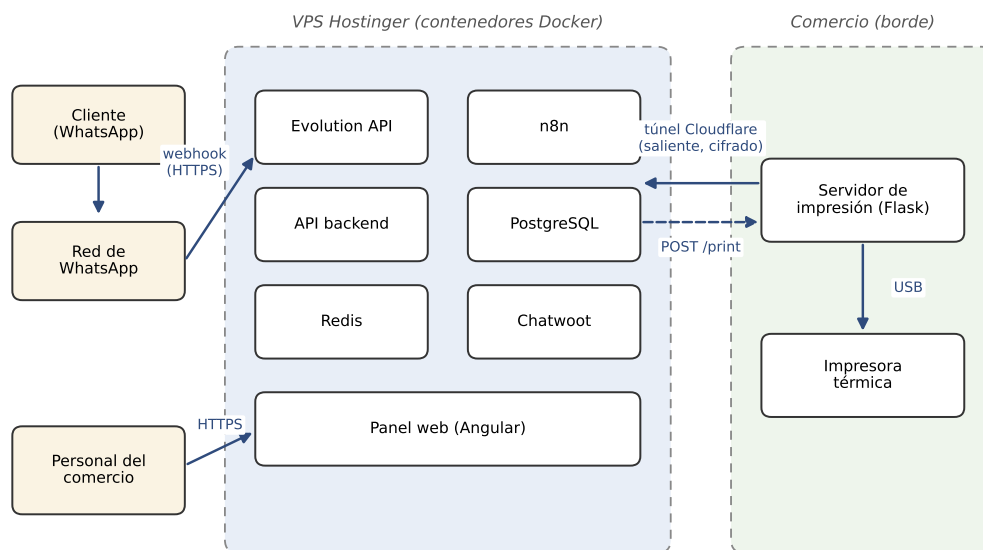


FIGURA 3.3. Topología de despliegue del sistema.

El tercero es el tramo entre la nube y el borde, resuelto mediante el túnel de Cloudflare.

3.3.2. Vinculación con el nodo de borde

La decisión de diseño más relevante de esta capa es el sentido en que se establece la conexión con el comercio. En lugar de exponer el servidor local a Internet mediante la apertura de puertos en el enrutador del negocio, el túnel se establece con una conexión saliente cifrada iniciada desde el propio nodo de borde.

Esta elección tiene tres consecuencias prácticas. La instalación no requiere configurar el equipamiento de red del local, condición decisiva en un comercio sin personal técnico. La red interna del negocio no queda accesible desde el exterior, lo que reduce la superficie de ataque. Y el vínculo se restablece de forma automática tras una interrupción, sin intervención manual.

3.3.3. Formato de los mensajes intercambiados

Los datos que circulan entre los componentes se serializan en formato JSON (del inglés, *JavaScript Object Notation*). El agente de inteligencia artificial produce una estructura fija, que constituye el contrato interno del sistema. El código 3.1 muestra su formato.

```
{
  "is_order": true,
  "items": [
    { "nombre": "Papa", "cantidad": 2, "unidad": "kg" }
  ],
  "aclaraciones": "bien lavadas",
  "productos_no_encontrados": []
}
```



```
}
```

CÓDIGO 3.1. Estructura del pedido devuelto por el agente de interpretación.

El campo `is_order` permite descartar los mensajes que no constituyen un pedido, como saludos o consultas de precio. El campo `items` contiene los productos reconocidos, con la denominación exacta del catálogo. El campo `aclaraciones` recoge las instrucciones y observaciones del cliente. El campo `productos_no_encontrados` registra las menciones que no pudieron asociarse a ningún artículo del catálogo, en lugar de descartarlas en silencio. Esta última decisión resultó importante para la trazabilidad: un producto solicitado y no reconocido queda registrado y visible para el personal, en vez de desaparecer del pedido.

El segundo contrato relevante es el del servidor de impresión. El código 3.2 muestra la petición que la API envía al nodo de borde.

```
POST /print
X-Printer-Token: <token>

{
  "text": "PEDIDO\n...",
  "printer_name": "Hasar_USB",
  "cut": true,
  "text_size": "double_width"
}
```

CÓDIGO 3.2. Petición de impresión enviada al servidor local.

El campo `text` contiene el ticket ya compuesto. Los campos restantes controlan el destino y el formato de impresión: el nombre de la impresora instalada, el corte automático del papel al finalizar y el tamaño de la tipografía. De manera alternativa, la petición admite una dirección IP y un puerto, para el caso en que la impresora se conecte por red. Esa previsión permite trasladar la impresora dentro del local sin modificar el código, tal como se anticipó en el capítulo 2.

El servidor de impresión resuelve el envío según la plataforma sobre la que se ejecuta. En Windows utiliza la interfaz de impresión del sistema operativo en modo directo, lo que evita que el controlador reinterprete el contenido y preserva los comandos ESC/POS. En sistemas Linux delega en el sistema de impresión estándar, y ante una impresora de red abre una conexión directa al puerto correspondiente. Esta triple implementación permitió desarrollar y ensayar el componente fuera del entorno del comercio.

3.4. Capa de aplicación y procesamiento

Esta capa concentra la interpretación de los pedidos, su validación, la resolución de precios y su persistencia. Es la de mayor complejidad lógica y la que resuelve el problema central del trabajo.

3.4.1. Interpretación mediante agentes de inteligencia artificial

La interpretación se implementó como un agente de inteligencia artificial orquestado en n8n. El agente recibe cuatro insumos: el texto agrupado del cliente, sus

datos de contacto, el catálogo vigente de productos y la lista de unidades admitidas. Los dos últimos no están fijados en el flujo, sino que se consultan a la API en cada ejecución mediante recursos internos previstos para ese fin. Esta decisión evita la duplicación del catálogo: el personal lo administra desde el panel y el agente siempre opera con la versión actualizada.

El agente cuenta además con memoria de ventana, cuya clave de sesión se construye a partir del teléfono del cliente. De este modo, cada conversación mantiene su contexto de forma independiente.

El comportamiento del agente se define mediante un mensaje de sistema, es decir, la instrucción permanente que delimita su rol y sus reglas antes de recibir cada mensaje del cliente. Su redacción constituyó una parte sustancial del desarrollo, porque de ella dependen tanto la calidad de la interpretación como la estabilidad del formato de salida. El código 3.3 reproduce sus fragmentos más representativos.

```
# ROL
Sos un asistente para una verduleria. Tu unica tarea es:
1. Determinar si el mensaje del cliente es un pedido de compra.
2. Extraer los productos, cantidades y unidades solicitados.

# CATALOGO
- Para el campo 'nombre' en cada item, usa exactamente el nombre
  del catalogo.
- Si el cliente menciona un producto que no esta en el catalogo,
  ponelo en 'productos_no_encontrados' (no en 'items').
- Si el cliente usa un apodo o abreviatura que corresponde a un
  producto del catalogo, usa el nombre del catalogo.

# SALIDA
Devuelve unicamente el JSON, sin texto adicional ni marcado.
```

CÓDIGO 3.3. Fragmentos del mensaje de sistema del agente de interpretación.

Tres criterios guiaron esa redacción. El primero es la delimitación estricta del rol, que evita que el agente responda consultas ajenas al pedido. El segundo es la restricción del formato de salida, indispensable para consumir la respuesta de forma programática. El tercero es el tratamiento explícito de los casos ambiguos, que impide que el modelo complete por su cuenta la información faltante.

3.4.2. Normalización semántica de productos

Un problema específico del dominio es la variabilidad con que los clientes nombran un mismo producto. Las denominaciones llegan en plural o singular, con diminutivos, con abreviaturas o con errores de tipeo. El sistema debe asociarlas al artículo correcto del catálogo del comercio.

La normalización se resolvió dentro del propio agente, mediante la entrega del catálogo completo como contexto y una instrucción explícita de correspondencia tolerante a variantes. El agente debe utilizar la denominación exacta del catálogo en el campo de salida. Cuando una mención no admite correspondencia razonable, se deriva al campo de productos no encontrados.

Se consideró como alternativa una normalización previa por reglas o por similitud de cadenas. Esa opción se descartó por dos razones. La primera es que las reglas exigen mantenimiento manual ante cada producto nuevo o cada variante no prevista. La segunda es que la similitud de cadenas resuelve mal los casos en que la diferencia es semántica y no ortográfica, como una denominación regional que no comparte raíz con el nombre del catálogo. La estrategia adoptada traslada esa responsabilidad al modelo, que dispone del catálogo completo y del contexto del mensaje.

3.4.3. Recursos de la interfaz de programación

La API se organizó por recursos, con un prefijo común y un archivo de rutas por entidad. En la tabla 3.1 se resumen los grupos de recursos y su mecanismo de acceso.

TABLA 3.1. Grupos de recursos de la interfaz de programación.

Recurso	Función	Acceso
/auth	Autenticación de personas	Público
/pedidos	Creación, consulta y edición	Clave de servicio o testigo
/productos	Catálogo de productos	Testigo (catálogo: clave)
/unidades	Unidades de medida	Testigo (catálogo: clave)
/categorias	Categorías de productos	Testigo, rol administrador
/clientes	Clientes y listas de precios	Testigo
/precios_default	Precios generales	Testigo
/print-logs	Historial de impresiones	Testigo
/metricas	Indicadores de operación	Testigo
/usuarios	Usuarios del sistema	Testigo, rol administrador

La creación de un pedido desde el flujo automático y su creación desde el panel se resolvieron con recursos distintos, aunque compartan la lógica de persistencia. La separación responde a que difieren en su origen, en su mecanismo de autenticación y en la validación aplicada: el flujo entrega productos ya normalizados por el agente, mientras que el panel recibe datos ingresados por una persona.

3.4.4. Modelo de datos

La persistencia se resolvió sobre PostgreSQL, con acceso mediante el ORM Prisma. El esquema se controla por migraciones versionadas, lo que permite reproducir la estructura de la base en cualquier entorno. En la figura 3.4 se presenta el diagrama entidad-relación con las entidades principales.

El núcleo del modelo lo constituyen las entidades `clientes`, `pedidos` e `items_pedido`. Un cliente se identifica por su teléfono, con restricción de unicidad, y acumula sus pedidos. Cada pedido registra la fecha, el estado, las aclaraciones y el mensaje original recibido por WhatsApp. Este último campo resulta clave para la trazabilidad, ya que permite contrastar el pedido estructurado contra el texto que le dio origen.

La entidad `items_pedido` presenta una particularidad de diseño. Junto a la referencia al producto del catálogo conserva un campo con la denominación original empleada por el cliente. Cuando la referencia al catálogo queda vacía, el ítem corresponde a un producto no reconocido. Esos ítems no se descartan: se conservan,

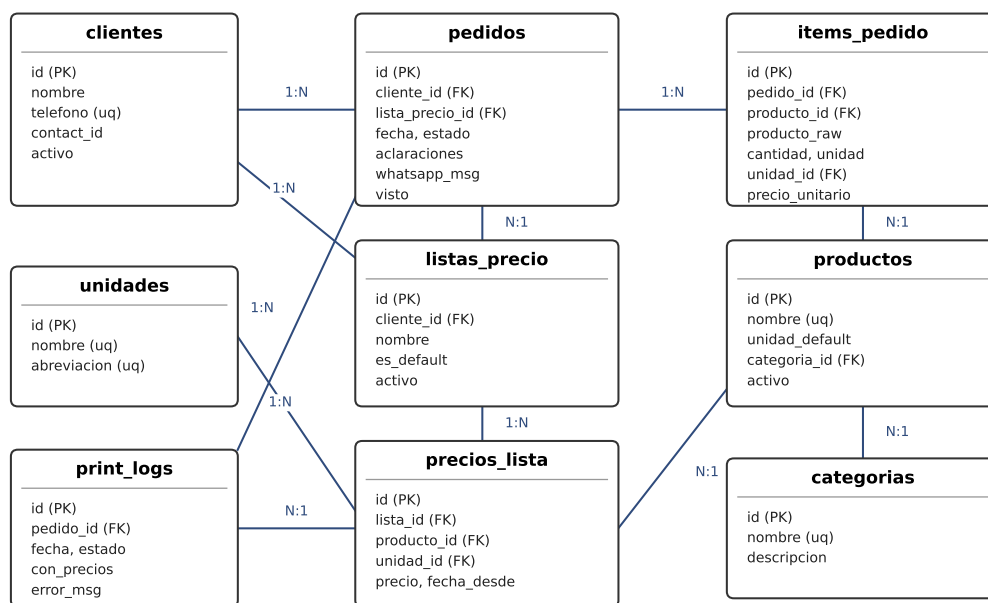


FIGURA 3.4. Diagrama entidad-relación de la base de datos.

se imprimen en una sección separada del ticket y se destacan visualmente en el panel para que el personal decida cómo proceder.

El estado del pedido se modeló como una enumeración con cuatro valores: recibido, en preparación, listo y entregado. La enumeración impide que la base admita estados no previstos y ordena el recorrido operativo del pedido dentro del comercio.

El modelo contempla además dos operaciones propias del uso real. La primera es la fusión de pedidos: cuando un cliente envía un pedido complementario poco después del primero, el personal puede unificarlos desde el panel. El modelo conserva la referencia al pedido resultante y la marca temporal de la operación, lo que permite deshacerla sin pérdida de información. La segunda es la marca de pedido visto, que distingue los pedidos ya revisados por el personal de los que aún esperan atención.

3.4.5. Resolución de precios

La gestión de precios responde a una característica del negocio: no todos los clientes pagan lo mismo. El comercio atiende tanto a consumidores finales como a otros comercios, con condiciones diferenciadas.

El modelo resuelve esa necesidad mediante listas de precios. Cada cliente puede tener varias listas, y una de ellas se designa como predeterminada. El precio de un ítem se resuelve por orden de prioridad: primero la lista indicada de forma explícita en el pedido, luego la lista predeterminada del cliente y, por último, el precio general del catálogo. Los precios incorporan además vigencia temporal, con fechas de inicio y de fin, lo que permite conservar el histórico sin perder el valor aplicado en cada momento.

Un diseño alternativo, con un único precio por cliente y producto, habría resultado más simple. Se descartó porque impide representar acuerdos comerciales distintos para un mismo cliente, como una condición mayorista y otra minorista, y porque no conserva el histórico de precios necesario para las métricas de facturación.

3.4.6. Generación del ticket

El ticket no se almacena en la base de datos, sino que se compone en el momento de la impresión a partir de los ítems del pedido y de sus aclaraciones. Esta decisión evita la duplicación de información y garantiza que una reimpresión refleje el estado actual del pedido, incluidas las correcciones realizadas por el personal.

La impresión admite dos variantes, con precios y sin precios. El flujo automático imprime sin precios, porque el ticket cumple allí la función de orden de armado. Desde el panel, en cambio, el personal elige la variante al momento de imprimir, según entregue el comprobante al cliente o lo destine al armado interno.

Cada intento de impresión se registra en una entidad específica, con su resultado, la variante utilizada y el mensaje de error en caso de falla. Ese registro cumple dos funciones: permite auditar qué se imprimió y cuándo, y habilita el seguimiento de los errores hasta su resolución. Los pedidos con impresión fallida se destacan en el panel y permanecen señalados hasta que el personal los marca como resueltos, de modo que una falla de la impresora no derive en un pedido sin atender.

3.5. Interfaz de usuario y visualización

El sistema presenta dos interfaces hacia el personal del comercio: el ticket impreso y el panel web de control.

3.5.1. El ticket impreso

El ticket es la salida física del sistema y el insumo directo para el armado del pedido. Su formato es fijo. Incluye un encabezado con los datos del cliente, la lista de productos con cantidades y unidades, una sección separada para los productos no reconocidos, un espacio para las aclaraciones y la fecha y hora del pedido. En la figura 3.5 se muestra un ticket generado por el sistema.

El diseño del ticket estuvo condicionado por el medio. El ancho útil del papel es de ochenta milímetros, lo que limita la cantidad de caracteres por línea y obliga a priorizar la información. Por ese motivo, el servidor de impresión admite distintos tamaños de tipografía, que se aplican mediante comandos ESC/POS. Los datos que el personal necesita leer a distancia, en particular el nombre del cliente, se imprimen en tamaño ampliado, mientras que el detalle de los productos se mantiene en tamaño normal para aprovechar el ancho disponible.

3.5.2. El panel web de control

El panel se desarrolló con Angular y la biblioteca de componentes Angular Material. Concentra la operación diaria y la administración del sistema. El acceso está protegido por autenticación, y las vistas de administración quedan restringidas según el rol de la persona que ingresa.

PEDIDO
Fecha y hora:
30/06/2026 01:43:18
Cliente: [REDACTED]
Tel: [REDACTED]

- Cebolla: 1 kg

- Batata: 1 kg

- Brócoli: 1 u.
\$3.000/u. \$3.000

- Tomate perita: 1 kg
\$25.000/kg \$25.000

- Morrón rojo: 1 u.

- Cebolla: 1 kg

- Batata: 1 kg

- Ají picante: 1 kg
\$14.554/kg \$14.554

- Morrón rojo: 2 u.
\$500/u. \$1.000

- Tomate perita: 1.5 kg
\$25.000/kg \$37.500

FUERA DE CATALOGO:

- Zapallo paraguayo: 1
a.

TOTAL: \$81.054

ACLARACIONES:

FIGURA 3.5. Ticket generado por el sistema e impreso en el comercio.

Tablero principal

El tablero es la vista de entrada al sistema y resume el estado de la operación del día. En la figura 3.6 se presenta esa vista.

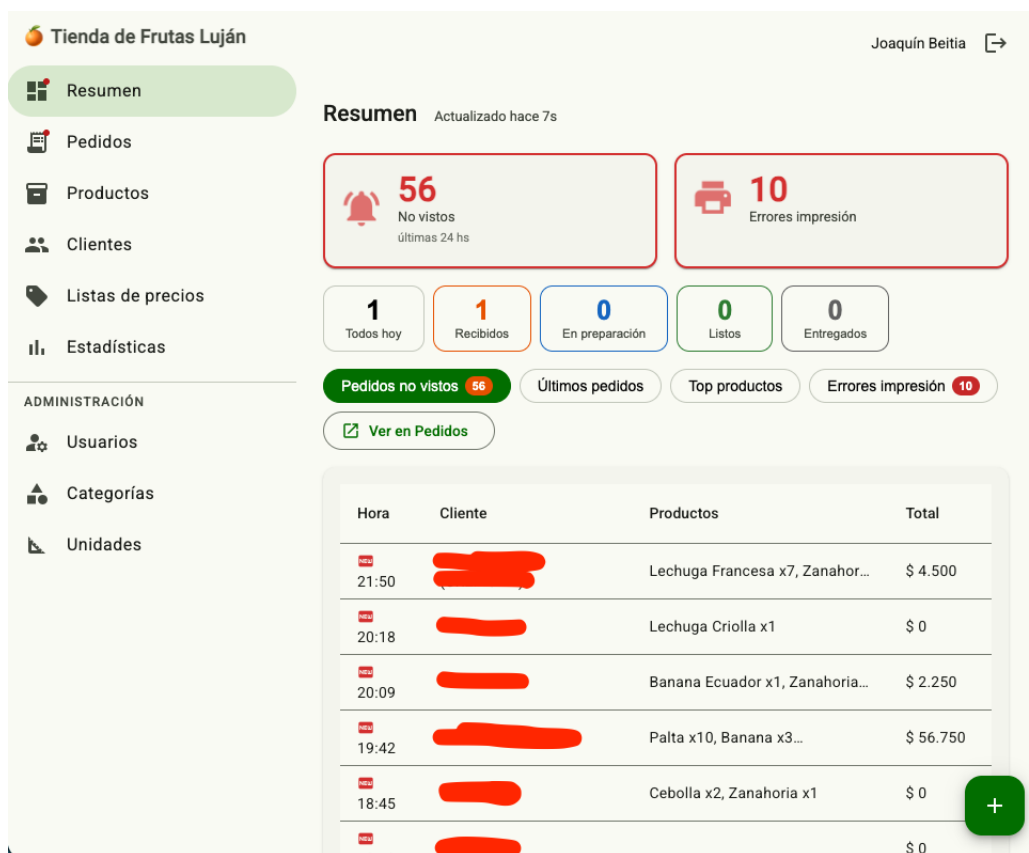


FIGURA 3.6. Tablero principal del panel de control.

El tablero agrupa cuatro bloques de información: los pedidos aún no revisados por el personal, los pedidos recibidos en los últimos minutos, los productos más solicitados en el día y los pedidos con errores de impresión pendientes de resolución. Cada bloque enlaza con la vista de pedidos, con los filtros ya aplicados, de modo que una alerta deriva de forma directa en la acción correspondiente.

Gestión de pedidos

La vista de pedidos es la de mayor uso durante la jornada. Permite filtrar por estado, por rango de fechas, por pedidos no vistos y por errores de impresión. En la figura 3.7 se muestra el listado.

El listado admite operaciones sobre varios pedidos a la vez, como el cambio de estado o la marca de revisado, pensadas para los momentos de mayor demanda. El detalle de cada pedido se abre en un diálogo que concentra la operación individual, según se observa en la figura 3.8.

Desde ese diálogo, el personal puede corregir las cantidades y las unidades, agregar o quitar ítems, consultar el mensaje original recibido por WhatsApp, acceder al hilo de la conversación e imprimir o reimprimir el pedido. Los ítems que no pudieron asociarse al catálogo se destacan de forma visual, lo que dirige la atención del personal hacia los casos que requieren decisión.

Pedidos Actualizado hace 3s

☐	Hora	Cliente	Productos	Total	Estado	Visto
☐	20/07 21:50	[REDACTED]	Lechuga Francesa x7, Zanahoria x4...	\$ 4.500	Recibido	☒
☐	20/07 20:18	[REDACTED]	Lechuga Criolla x1	—	Recibido	☒
☐	20/07 20:09	[REDACTED]	Banana Ecuador x1, Zanahoria x1...	\$ 2.250	Recibido	☒
☐	20/07 19:42	[REDACTED]	Palta x10, Banana x3...	\$ 56.750	Recibido	☒
☐	20/07 18:45	[REDACTED]	Cebolla x2, Zanahoria x1	—	Recibido	☒
☐	20/07 18:43	[REDACTED]		—	Recibido	☒
☐	20/07 18:16	[REDACTED]	Banana x2.5, Banana x3...	\$ 357.200	Recibido	☒
☐	20/07 17:22	[REDACTED]	Albahaca x0.2, Anco x28.4...	\$ 258.090	Recibido	☒
☐	20/07 17:21	[REDACTED]	Albahaca x2, Tomate Cherry x4	\$ 30.000	Recibido	☒
☐	20/07 16:46	[REDACTED]	Frutilla x0.5	—	Recibido	☒

FIGURA 3.7. Listado de pedidos con sus filtros de búsqueda.

Administración del catálogo y de los precios

El catálogo de productos, sus categorías y las unidades de medida se administran desde el panel. Esta gestión resulta crítica para el funcionamiento del sistema, ya que el catálogo es el insumo que el agente utiliza para normalizar las denominaciones. Un producto ausente del catálogo se traduce de inmediato en ítems no reconocidos en los pedidos.

La administración de precios se resolvió en una vista específica, con el listado de listas del cliente a un lado y la tabla de precios editable al otro. En la figura 3.10 se presenta esa vista.

Métricas de operación

Las métricas se representan con la biblioteca Chart.js. La vista incluye el tráfico de pedidos por día y hora, la evolución de la facturación por producto y la distribución por categoría. En la figura 3.11 se muestra esa vista.

La consulta admite distintos rangos temporales, y la granularidad de la serie se ajusta de forma automática al rango solicitado: diaria hasta una semana, semanal hasta tres meses y mensual para períodos mayores. Este ajuste evita gráficos ilegibles por exceso de puntos. El tráfico por día y hora resultó de utilidad directa para el negocio, ya que expone los momentos de mayor demanda y permite anticipar la asignación de personal.

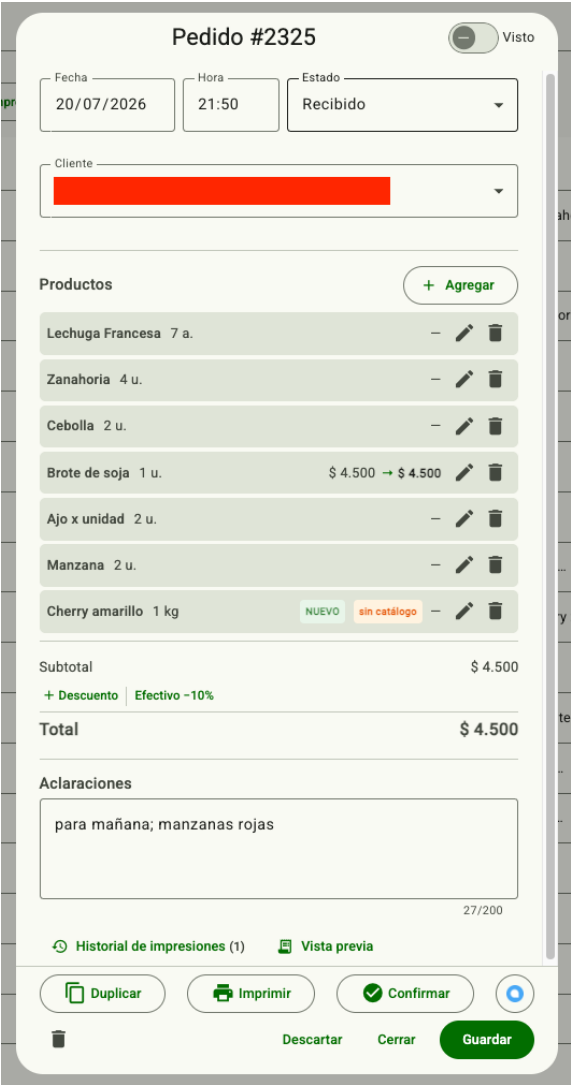


FIGURA 3.8. Detalle de un pedido, con edición de ítems y acceso al mensaje original.

Productos

Buscar producto

Categoría

Todas

☐ Mostrar inactivos

Exportar precios

Importar precios

Nombre	Categoría	Unidad	Precios	Activo
Aceite	Varios	unidad	\$9.500/unidad	☒
Acelga	Hojas	cajon	\$1.800/atado \$15.000/cajon	☒
Aji Picante	Verduras	cajon	\$-/cajon \$10.000/kg	☒
Aji Vinagre	Verduras	cajon	\$-/cajon \$25.000/cajon \$4.000/kg	☒
Ajo	Varios	riestra	\$1.500/unidad \$50.000/riestra	☒

FIGURA 3.9. Administración del catálogo de productos.

Cliente: Diego Empleado – 5492323522266

Listas de precios

EJEMPLO  

+ Nueva lista

EJEMPLO
★ Lista default

 Exportar CSV  Importar CSV

Exportá, editá en Google Sheets y reimportá. El archivo es la fuente de verdad: una celda vacía elimina ese precio.

Producto	Unidad	Precio especial	Precio base	Diferencia	
Anco	bolsa	\$ 15.000	—	—	 
Acelga	unidad	\$ 1.000	—	—	 
Ajo	unidad	\$ 2.000	\$ 1.500	33%	 

Agregar precio

Producto Unidad \$ Precio

FIGURA 3.10. Administración de las listas de precios de un cliente.

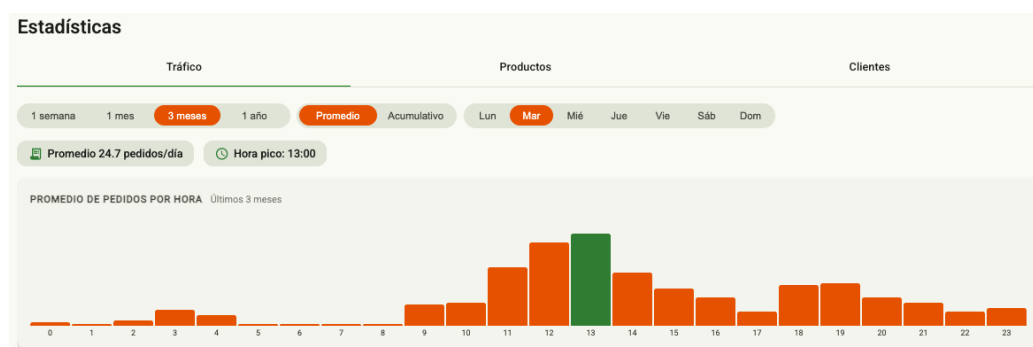


FIGURA 3.11. Vista de métricas de operación del comercio.

3.6. Seguridad del sistema

La seguridad se abordó en tres frentes: el control de acceso, la protección de las comunicaciones y la integridad de los datos.

3.6.1. Autenticación y control de acceso

El sistema distingue dos tipos de consumidores, con mecanismos de autenticación distintos. Las personas acceden al panel con credenciales y obtienen un testigo de sesión. Los servicios internos, como el flujo de n8n, no disponen de una sesión de usuario y se autentican con una clave de servicio. En la figura 3.12 se representan ambos mecanismos.

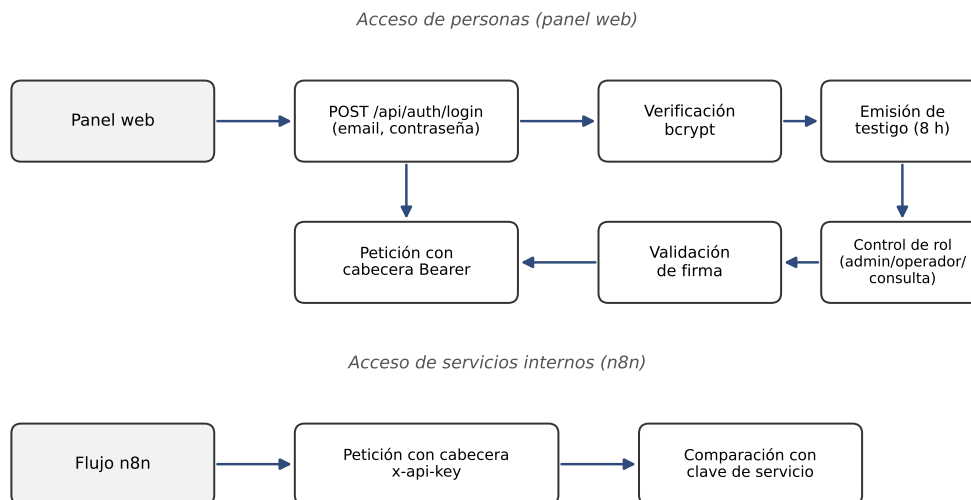


FIGURA 3.12. Mecanismos de autenticación para personas y para servicios internos.

Las contraseñas se almacenan cifradas mediante una función de derivación diseñada para ese fin, de modo que la base no conserva las claves en texto plano. Tras una validación exitosa, el sistema emite un testigo firmado con vigencia de ocho horas, que el panel adjunta en cada petición posterior. La vigencia se ajustó a la duración de una jornada comercial, lo que evita tanto la reautenticación durante el turno como la persistencia indefinida de una sesión abierta.

La autorización se implementó por roles. El sistema define tres: administrador, con acceso total; operador, restringido a la operación de pedidos; y consulta, limitado a la lectura. La verificación se resuelve con un componente intermedio que valida el rol antes de ejecutar cada operación, lo que centraliza la regla y evita repetirla en cada recurso. El panel replica esa restricción en sus rutas, de modo que las vistas de administración no resultan accesibles para los roles que no corresponden.

Los recursos internos, como la consulta del catálogo y la creación de pedidos desde el flujo, admiten únicamente la clave de servicio. El recurso de impresión

constituye una excepción deliberada, ya que acepta ambas formas de autenticación: la clave de servicio para la impresión automática y el testigo de sesión para la reimpresión desde el panel.

3.6.2. Protección de las comunicaciones

Todo el tráfico externo circula cifrado. La pasarela de mensajería entrega los mensajes por HTTPS y el vínculo entre la nube y el borde se resuelve mediante el túnel descrito en la sección 3.3.2, cuya conexión saliente cifrada evita exponer puertos entrantes en la red del comercio. El servidor de impresión exige, además, un testigo propio en cada petición, de modo que la sola llegada al servicio no habilita la impresión.

3.6.3. Gestión de credenciales e integridad de los datos

Las credenciales de los servicios externos y las claves de firma se gestionan mediante variables de entorno, fuera del código fuente. Los archivos que las contienen quedan excluidos del control de versiones, y el repositorio conserva únicamente valores de ejemplo. Este criterio evita que una credencial operativa quede registrada en el historial del repositorio.

En cuanto a la integridad, el modelo de datos define restricciones de unicidad sobre los campos que identifican a clientes, productos y unidades, y relaciones con integridad referencial entre las entidades. La eliminación de un pedido arrastra sus ítems y sus registros de impresión, lo que evita la persistencia de datos huérfanos. Redis se reservó de manera estricta para información temporal, de modo que una falla en la capa en memoria no compromete los registros del negocio.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] Shopify. *Shopify POS: sistema de punto de venta*. Shopify. URL: <https://www.shopify.com/pos> (visitado 06-07-2026).
- [2] Tiendanube. *¿Qué es Punto de Venta de Tiendanube?* Tiendanube. URL: https://ayuda.tiendanube.com/es_AR/pdv/que-es-punto-de-venta-de-tiendanube (visitado 06-07-2026).
- [3] n8n. *n8n: Workflow Automation*. n8n. URL: <https://n8n.io> (visitado 06-07-2026).
- [4] Zapier. *Zapier: Automation for Business*. Zapier. URL: <https://zapier.com> (visitado 06-07-2026).
- [5] Weisong Shi et al. «Edge Computing: Vision and Challenges». En: *IEEE Internet of Things Journal* 3.5 (oct. de 2016), págs. 637-646. DOI: [10.1109/JIOT.2016.2579198](https://doi.org/10.1109/JIOT.2016.2579198).
- [6] Mahadev Satyanarayanan. «The Emergence of Edge Computing». En: *Computer* 50.1 (ene. de 2017), págs. 30-39.
- [7] Liang Tong, Yong Li y Wei Gao. «A Hierarchical Edge Cloud Architecture for Mobile Computing». En: *IEEE INFOCOM 2016 – The 35th Annual IEEE International Conference on Computer Communications*. San Francisco, CA, USA: IEEE, abr. de 2016, págs. 1-9.
- [8] Microsoft. *Windows Server*. Microsoft. URL: <https://www.microsoft.com/en-us/windows-server> (visitado 20-07-2026).
- [9] Raspberry Pi Ltd. *Raspberry Pi*. Raspberry Pi Ltd. URL: <https://www.raspberrypi.com> (visitado 20-07-2026).
- [10] Compañía Hasar. *Impresora térmica P-HAS-181*. Grupo Hasar. URL: <https://compania.grupohasar.com/producto/p-has-181/> (visitado 13-07-2026).
- [11] Seiko Epson Corporation. *ESC/POS Command Reference for TM Printers*. Epson. URL: https://download4.epson.biz/sec_pubs/pos/reference_en/escpos/index.html (visitado 20-07-2026).
- [12] Roy T. Fielding, Mark Nottingham y Julian Reschke. *HTTP Semantics*. Request for Comments 9110. Internet Engineering Task Force, jun. de 2022. DOI: [10.17487/RFC9110](https://doi.org/10.17487/RFC9110). URL: <https://www.rfc-editor.org/info/rfc9110> (visitado 20-07-2026).
- [13] Roy Thomas Fielding. «Architectural Styles and the Design of Network-based Software Architectures». Tesis doct. University of California, Irvine, 2000.
- [14] OpenAPI Initiative. *OpenAPI Specification v3.1.0*. OpenAPI Initiative. 2021. URL: <https://spec.openapis.org/oas/v3.1.0.html> (visitado 20-07-2026).
- [15] Evolution API. *Evolution API: Open-source WhatsApp Integration API*. Evolution API. URL: <https://github.com/EvolutionAPI/evolution-api> (visitado 13-07-2026).
- [16] Meta. *WhatsApp Business Platform*. Meta Platforms. URL: <https://developers.facebook.com/docs/whatsapp> (visitado 06-07-2026).
- [17] Cloudflare. *Cloudflare Tunnel*. Cloudflare. URL: <https://www.cloudflare.com/products/tunnel/> (visitado 06-07-2026).

- [18] Hostinger. *VPS Hosting*. Hostinger. URL: <https://www.hostinger.com/vps-hosting> (visitado 13-07-2026).
- [19] Docker. *Docker: Accelerated Container Application Development*. Docker, Inc. URL: <https://www.docker.com> (visitado 06-07-2026).
- [20] OpenAI. *OpenAI Platform*. OpenAI. URL: <https://platform.openai.com> (visitado 06-07-2026).
- [21] Anthropic. *Claude*. Anthropic. URL: <https://www.anthropic.com/claude> (visitado 13-07-2026).
- [22] OpenAI. *Whisper: Robust Speech Recognition*. OpenAI. URL: <https://github.com/openai/whisper> (visitado 06-07-2026).
- [23] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. The PostgreSQL Global Development Group. URL: <https://www.postgresql.org> (visitado 06-07-2026).
- [24] Redis. *Redis: The Real-time Data Platform*. Redis. URL: <https://redis.io> (visitado 06-07-2026).
- [25] Pallets. *Flask: Web Development, One Drop at a Time*. Pallets. URL: <https://flask.palletsprojects.com> (visitado 06-07-2026).
- [26] Google. *Angular*. Google. URL: <https://angular.dev> (visitado 20-07-2026).
- [27] Chart.js. *Chart.js: Simple yet flexible JavaScript charting library*. Chart.js. URL: <https://www.chartjs.org> (visitado 20-07-2026).